CHAPTER III

REQUIREMENTS SPECIFICATIONS

3.1 Functional Requirements

The functional requirements describe what the system is expected to do in a real university environment. They focus on the main features that will help manage daily activities, support users, and ensure everything runs smoothly across the institution.

The system is built around three main areas:

1. Access Control

The system will manage user accounts created by the institution and ensure that only authorized users can access it. Each user will log in through a secure session, and in some cases, they may be required to change their password. Access to different features will depend on the user's role (such as student, teacher, or admin), so everyone only sees what they are supposed to.

2. Academic Operations

This part of the system handles all academic-related activities. It includes managing study programs, courses, and course offerings, as well as student registration and scheduling. The system will also support tracking attendance, managing grades, and monitoring student progress throughout their studies.

3. Campus Services

In addition to academics, the system will provide features that improve student life and communication. These include managing finances, organizing clubs and events, sending announcements and notifications, and allowing students to interact with support services when needed.

Overall, these features work together to create a system that is practical, easy to use, and helpful for both students and staff in their everyday tasks.Below is a detailed list of the functional requirements that define how these features are expected to operate within the system. Each requirement outlines a specific capability, its priority, and the conditions under which it is considered successfully implemented.

Req#	Requirement	Comments	Priority	Date	SME Reviewed/Approved
FR_01	Account Provisioning	The system should allow admins to create student, teacher, and staff accounts using institutional emails.	High	12/04/26	Fabio Hysa Kleart Adri Arbri Gaba Helio Myrteza Fabjan Likaj Erli Halilaj
FR_02	Role-Based Access Control	Users should only see features and data based on their role (student, teacher, admin).	High	12/04/26	Fabio Hysa Kleart Adri Arbri Gaba Helio Myrteza Fabjan Likaj Erli Halilaj
FR_03	First-Time Password Change	New users must change their temporary password before accessing the system.	High	12/04/26	Fabio Hysa Kleart Adri Arbri Gaba Helio Myrteza Fabjan Likaj Erli Halilaj
FR_04	Password Reset	Users should be able to request and securely reset their passwords.	High	12/04/26	Fabio Hysa Kleart Adri Arbri Gaba Helio Myrteza Fabjan Likaj Erli Halilaj

FR_05	Academic Structure Management	The system should manage departments, programs, courses, terms, and offerings.	High	12/04/26	Fabio Hysa Kleart Adri Arbri Gaba Helio Myrteza Fabjan Likaj Erli Halilaj
FR_06	Teacher Assignment	Admins should assign teachers to course offerings.	High	12/04/26	Fabio Hysa Kleart Adri Arbri Gaba Helio Myrteza Fabjan Likaj Erli Halilaj
FR_07	Course Catalog & Recommendations	Students should browse courses and see recommendations during registration.	Medium	12/04/26	Fabio Hysa Kleart Adri Arbri Gaba Helio Myrteza Fabjan Likaj Erli Halilaj
FR_08	Course Registration	Students should add/drop courses based on rules like prerequisites and capacity.	High	12/04/26	Fabio Hysa Kleart Adri Arbri Gaba Helio Myrteza Fabjan Likaj Erli Halilaj
FR_09	Timetable View	Students should view their schedule based on registered courses.	Medium	12/04/26	Fabio Hysa Kleart Adri Arbri Gaba Helio Myrteza

					Fabjan Likaj Erli Halilaj
FR_10	Attendance Management	Teachers should create sessions and mark student attendance.	High	12/04/26	Fabio Hysa Kleart Adri Arbri Gaba Helio Myrteza Fabjan Likaj Erli Halilaj
FR_11	Grading System	Teachers should manage grades and publish results.	High	12/04/26	Fabio Hysa Kleart Adri Arbri Gaba Helio Myrteza Fabjan Likaj Erli Halilaj
FR_12	Student Academic View	Students should view grades, attendance, and academic progress.	High	12/04/26	Fabio Hysa Kleart Adri Arbri Gaba Helio Myrteza Fabjan Likaj Erli Halilaj
FR_13	Notifications Inbox	Users should receive and manage system notifications and alerts.	High	12/04/26	Fabio Hysa Kleart Adri Arbri Gaba Helio Myrteza Fabjan Likaj Erli Halilaj

FR_14	Announcements & Events	Admins should publish news and events visible to students.	High	12/04/26	Fabio Hysa Kleart Adri Arbri Gaba Helio Myrteza Fabjan Likaj Erli Halilaj
FR_15	Clubs & Participation	Students should join clubs and events through the system.	Medium	12/04/26	Fabio Hysa Kleart Adri Arbri Gaba Helio Myrteza Fabjan Likaj Erli Halilaj
FR_16	Club Management	Admins should manage clubs and review membership requests.	Medium	12/04/26	Fabio Hysa Kleart Adri Arbri Gaba Helio Myrteza Fabjan Likaj Erli Halilaj
FR_17	Finance Management	Admins should issue invoices, record payments, and manage holds.	High	12/04/26	Fabio Hysa Kleart Adri Arbri Gaba Helio Myrteza Fabjan Likaj Erli Halilaj
FR_18	Student Finance View	Students should view balances and request financial support.	Medium	12/04/26	Fabio Hysa Kleart Adri Arbri Gaba Helio Myrteza

					Fabjan Likaj Erli Halilaj
FR_19	Audit Logging	The system should track important actions like login and account changes.	High	12/04/26	Fabio Hysa Kleart Adri Arbri Gaba Helio Myrteza Fabjan Likaj Erli Halilaj
FR_20	User Account Management	Admins should activate, suspend, or reset accounts easily.	High	12/04/26	Fabio Hysa Kleart Adri Arbri Gaba Helio Myrteza Fabjan Likaj Erli Halilaj
FR_21	Teacher Workspace	Teachers should access courses, attendance, grading, and messages.	High	12/04/26	Fabio Hysa Kleart Adri Arbri Gaba Helio Myrteza Fabjan Likaj Erli Halilaj
FR_22	Reporting Dashboard	Admins should view reports for academic, finance, and system usage.	Medium	12/04/26	Fabio Hysa Kleart Adri Arbri Gaba Helio Myrteza Fabjan Likaj Erli Halilaj

FR_23	AI Chat Support	The system should store AI chat sessions for student support.	High	12/04/26	Fabio Hysa Kleart Adri Arbri Gaba Helio Myrteza Fabjan Likaj Erli Halilaj
FR_24	API Integration	The system should provide stable backend APIs for all modules.	High	12/04/26	Fabio Hysa Kleart Adri Arbri Gaba Helio Myrteza Fabjan Likaj Erli Halilaj

3.2 Nonfunctional Requirements

Non-functional requirements define the quality envelope within which CIS must operate. These constraints are essential for institutional trust, continuity, and operational viability.

3.2.1 Product Requirements

The user interface of the Campus Information System should be accessible through common modern browsers so that users can access it from desktop computers, tablets, and mobile devices. Since the system is intended for students, teachers, administrators, and other institutional users, the interface must be simple, role-appropriate, and consistent across modules. Based on the current project scope, the main user interface can be grouped into the following major interfaces: academic access interface, student workspace, teacher workspace, administrator workspace, and campus services interface.

3.2.1.1 User Interface Requirements.

The user interface of the Campus Information System should be accessible through common modern browsers so that users can access it from desktop computers, tablets, and mobile devices. Since the system is intended for students, teachers, administrators, and other institutional users, the interface must be simple, role-appropriate, and consistent across modules. Based on the current project scope, the main user interface can be grouped into the following major interfaces: academic access interface, student workspace, teacher workspace, administrator workspace, and campus services interface.

1. ***Authentication Interface***

   The authentication interface should provide the entry point for all institutional users. Since account creation is managed by the institution, the system does not rely on public self-registration, but it must still provide secure login, password change, and password reset functions .

   The login interface should contain:

- A field for institutional email address or username.
- A field for password entry.
- A “Forgot Password?” link for controlled password reset.
- A login button that validates credentials and redirects the user to the correct role dashboard.

- A forced password change screen for first-time users using temporary credentials.

2. Student Interface

The student interface should be designed as a self-service workspace where students can manage their academic and campus-related activities. It should provide quick access to personal academic data, course registration, grades, attendance, finance information, clubs, announcements, and notifications .

The student workspace should include:

- A dashboard showing timetable, notices, alerts, and summary information.
- A profile section for personal and academic information.
- A registration section for browsing and adding or dropping eligible courses.
- A grades and attendance section.
- A finance section for invoices, payments, and holds.
- A campus services section for clubs, events, announcements, and inbox items.

3. Teacher Interface

The teacher interface should support instructional workflows and allow academic staff to access only the offerings assigned to them. This interface should prioritize course rosters, attendance sessions, grading activities, and course communication .

The teacher workspace should include:

- A dashboard showing assigned offerings.
- A roster view for enrolled students.
- Attendance creation and attendance marking features.
- Grade component and final grade management.
- Notifications and academic alerts related to teaching responsibilities.

4. Administrator Interface

The administrator interface should provide broad institutional control over users, academic structures, finance operations, and campus communication. Since the administrator is responsible for provisioning accounts and managing institutional workflows, this interface must present high-level visibility and management functions .

Admin Interface Area	Description
User Management	Create, activate, suspend, reset, and manage student, teacher, and staff accounts
Academic Structure	Manage departments, programs, terms, courses, offerings, and teacher assignments
Finance Management	Issue invoices, record payments, and apply or release financial holds
Communication	Publish announcements, create events, and manage campus-wide notifications
Reporting	View institutional summaries for academic, finance, user, and engagement modules
Audit Oversight	Review traceable records of critical and security-sensitive actions

5. Campus Services and Communication Interface

This interface should support the wider student experience beyond core academics. It includes clubs, join requests, announcements, events, event participation, support requests, recommendations, and notifications.

Service Interface Feature	Purpose
Announcements	Publish institutional notices to target users
Events	Advertise academic or campus events and allow participation tracking
Clubs	Allow browsing of clubs and submission of membership requests
Notifications	Deliver reminders, warnings, and system messages

Support	Allow students to submit service or support-related requests
AI Chat Support	Store guided support interactions where institutional policy permits

3.2.1.2 Learnability

Learnability is an important quality requirement for the Campus Information System because the system will be used by users with different technical backgrounds. Students, teachers, and administrative staff should be able to understand the platform quickly and perform their core tasks without formal training. Since the project is intended as a centralized institutional platform, the interface should reduce confusion by using clear labels, predictable navigation, and role-based workspaces .

The learnability requirements are:

- Students should be able to understand how to check schedules, register courses, view grades, and review finance information after minimal use.
- Teachers should be able to learn attendance and grading workflows quickly through dedicated screens.
- Administrators should be able to understand management functions through grouped control panels and dashboard summaries.
- In case of an error, the system should display a clear and understandable message so that the user knows what went wrong.
- The terminology used in the platform should match university language such as course, offering, registration, grade, payment, hold, and notification.

3.2.1.3 Accessibility

Accessibility ensures that the system can be used by a wider range of users and from a wider range of devices. Since CIS is a web-based system, users should be able to access it from modern browsers without platform-specific installation. The interface should support accessible structure, readable text, and keyboard-operable workflows, which is also consistent with the non-functional expectations already stated in the project document.

Accessibility Requirement	Description
Browser Accessibility	The platform should work on commonly used modern browsers
Device Accessibility	The system should support desktop, tablet, and mobile-responsive access
Readability	Text, labels, and actions should remain readable with appropriate contrast
Keyboard Access	Primary workflows should be operable by keyboard
Semantic Structure	Forms and controls should use accessible labels and logical grouping

3.2.1.4 Efficiency

Efficiency is important because the system will support frequent daily operations such as viewing dashboards, opening course lists, recording attendance, publishing grades, and reviewing finance records. The platform should allow users to complete tasks without noticeable delays under normal university usage. The uploaded requirements already state that common dashboard and list views should meet acceptable latency expectations, especially for routine queries .The efficiency requirements are:

- Typical dashboard and list pages should load within acceptable user-perceived response time.
- Common queries should aim for sub-2-second performance under normal institutional load.
- Students should be able to register courses without unnecessary page transitions.
- Teachers should be able to record attendance and grades efficiently from dedicated views.
- Administrators should be able to manage users, academic structures, and finance operations from centralized interfaces.
- Backend API design should avoid unnecessary repeated requests and redundant database access.

3.2.1.5 Memorability

Memorability refers to how easily users can return to the system and continue using it after a period of not using it. Since university systems are used repeatedly but not always continuously in the same way by every role, CIS should keep its structure stable and predictable.

The memorability requirements are:

- The system should use consistent menu positions and naming across modules.
- Dashboards should preserve familiar layouts for each role.
- Students should easily remember where to find course registration, grades, attendance, and invoices.
- Teachers should easily remember where to find rosters, attendance sessions, and grading tools.
- Administrators should be able to return to the system and quickly identify management and reporting areas.

3.2.1.6 Errors

Error prevention and error handling are especially important in a university platform because the system manages official academic, identity, and finance data. The project specification already emphasizes validation, predictable responses, and prevention of contradictory records through relational constraints and backend enforcement.

Error Requirement	**Description**
Validation	All critical forms should validate input before data is submitted
Safe Failure	System failures should return understandable messages without exposing sensitive internals
Record Integrity	Invalid operations should not create duplicate or contradictory academic records
Permission Errors	Users should be informed when an action is blocked by authorization rules
Business Rule Errors	Students should be informed if registration fails because of prerequisites, capacity, or finance holds

3.2.1.7 Satisfaction

Satisfaction refers to the degree to which the system supports smooth and understandable user experience. A satisfying university system is one that helps users complete tasks with confidence and without confusion.

The satisfaction requirements are:

- The system should be user-friendly for students, teachers, and administrators.
- Core workflows such as registration, attendance submission, and invoice review should require minimal effort.
- Users should feel confident that their actions have been completed through visible confirmations and updated status.
- Notifications and alerts should help users remain informed without overwhelming them.
- The platform should reduce fragmentation by centralizing multiple campus services in one place .

3.2.1.8 Capacity

Capacity refers to the system's ability to store and process the amount of data expected in an institutional setting. Since CIS includes academic history, finance records, attendance, grades, notifications, support data, and possibly AI chat histories, the system must be designed to support large but structured datasets .

The capacity requirements are:

- The database should support thousands of students, teachers, users, courses, and transactions.
- Historical academic records should remain preserved for governance and reporting purposes.
- Notifications, announcements, and event records should be stored without degrading performance.
- Audit records should remain queryable for security and governance reviews.
- If AI chat history is retained, it should follow retention and privacy controls.

3.2.2 Organizational Requirements

3.2.2.1 Availability

Because CIS is a web-based institutional platform, it should be available throughout the day to support academic and administrative work. Availability is especially important during registration periods, grading periods, and finance-related deadlines. The uploaded document already highlights backup, restart, and recovery posture as a high-priority concern .

The availability requirements are:

- The system should be available 24 hours per day, 7 days per week, except for scheduled maintenance.
- Planned maintenance should be performed in periods of lower institutional activity where possible.
- The platform should support backup and recovery without data corruption.
- Routine restarts or deployments should minimize service interruption.
- Users should be informed in advance when planned maintenance is expected.

3.2.2.2 Latency

Latency is influenced by the internet connection, backend implementation quality, API design, and database performance. Since the system depends on a FastAPI backend and relational persistence, latency must remain acceptable even when data volume grows or concurrent usage increases.

Main Latency Factors	**Description**
Internet Connection	User-side network quality affects response time
Backend Processing	Complex business logic may increase request time
Database Size	Large record volumes may slow queries if not optimized
Query Design	Poorly designed queries can create bottlenecks
Peak Usage	Registration and grading periods may increase load

The system should therefore:

- Optimize common academic and dashboard queries.
- Use indexing and relational design effectively.
- Avoid unnecessary data loading on the frontend.
- Maintain stable response times under normal institutional load.

3.2.2.3 Monitoring

Monitoring is necessary to keep the system reliable and secure. Because CIS handles sensitive academic and finance workflows, the platform should support monitoring of system health, error conditions, and security-sensitive activity. This is consistent with the project's requirements for auditability and operational traceability .

The monitoring requirements are:

- Authentication attempts should be logged.
- Security-sensitive and governance-sensitive actions should be traceable.
- Operational errors should be logged for diagnosis.
- System uptime and service health should be monitored.
- Audit records should support administrative review and incident investigation.

3.2.2.4 Maintenance

Maintenance refers to the ability to update, repair, and improve the system over time. Since the project is already structured around frontend, backend, and database separation, it should remain maintainable and modular as the system evolves .The maintenance requirements are:

- Frontend, backend, and database layers should evolve without full redesign.
- New modules should be added without breaking stable existing modules.
- Schema migrations should be controlled and reversible where practical.
- Scheduled maintenance should be communicated to users.
- The codebase should be organized in a way that supports testing and future enhancement.

3.2.2.5 Operations

Operational requirements define how the system is deployed and run in practice. Based on the uploaded document, CIS already assumes a modular web architecture with React, FastAPI, and relational SQL persistence .

Operational Area	Requirement
Deployment	The application should support containerized or repeatable deployment
Configuration	Environment-based configuration should be used for secrets and runtime settings
Backend Services	The backend should expose authenticated REST endpoints for core modules
Database Operations	The relational database should support transactional integrity and reporting queries
Recovery	Backup and restoration procedures should be part of operations planning

3.2.2.6 Standards Compliance

CIS should follow recognized software and institutional standards. Since it manages university data, it should also align with privacy and governance expectations relevant to Albania and broader European data protection principles.The standards compliance requirements are:

- The API layer should follow REST-based design principles.
- The system should use structured JSON communication between frontend and backend.
- Personal data handling should align with institutional privacy obligations.
- The platform should align with Albanian personal data protection law and GDPR-oriented principles where applicable.
- Academic and finance data should be processed only for authorized institutional purposes.

3.2.2.7 Portability

Since CIS is a web-based platform, portability means that users should access it from different browsers and devices, and the system itself should be deployable in more than one infrastructure setting.

The portability requirements are:

- The platform should be accessible from desktop and mobile browsers.
- The system should not depend on a single proprietary environment.
- Deployment should support conventional server environments.
- The application should remain functional across supported browsers and devices.

3.2.3 External Requirements

3.2.3.1 Security

Security is one of the most important external requirements of the system because CIS handles institutional identity, grades, attendance, finance, and support information. The uploaded document already defines token verification, server-side authorization, password hashing, lockout controls, privacy restrictions, and auditability as mandatory or high-priority elements .

Security Requirement	**Description**
Verified Tokens	All authenticated API calls shall require verified tokens
Password Hashing	Passwords shall be stored only as secure hashes
Login Protection	Repeated failed attempts should trigger throttling or temporary lockout
Server-Side Authorization	Access control must not rely only on frontend behavior
Audit Logging	Sensitive and governance-relevant actions should be logged
Backup and Recovery	Data should be recoverable in the event of failure

3.2.3.2 Protection

Protection refers to the measures used to keep the system safe from misuse, invalid input, and unauthorized data exposure. The backend must enforce business rules and data integrity even if the client-side interface is bypassed, which is also clearly stated in the uploaded business rules section .

The protection requirements are:

- Passwords must never be stored in plain text.
- User input should be validated before database operations are performed.
- Backend rules should prevent unauthorized viewing or modification of records.
- Academic records, finance records, and audit records must remain protected from unapproved changes.
- Personally identifiable information should be visible only to explicitly authorized users.

3.2.3.3 Authorization and Authentication

Authentication confirms the user's identity, while authorization determines what the user is allowed to do. Since CIS is a multi-role university platform, role enforcement must be one of the strongest guarantees of the system .

The authorization and authentication requirements are:

- Users should authenticate through institutional credentials.
- New users should be forced to change temporary passwords on first login.
- Password reset tokens should be issued, validated, and consumed safely.
- Students should only access their own records.
- Teachers should only manage assigned offerings and related students.
- Administrators should access broader institutional controls according to their role.
- Sessions and tokens should expire according to secure configuration rules.

3.3 Domain Requirements

The domain requirements of CIS arise directly from the nature of a university information system. The system exists within a single institutional boundary where account creation is controlled internally and user roles determine functional scope. The backend is assumed to exist and to enforce academic, finance, identity, and engagement rules through authenticated services and relational data integrity .

The main domain requirements are:

- The system shall support academic structures such as departments, programs, terms, courses, and offerings.
- Students shall register only for offerings they are eligible to take.
- Teachers shall manage attendance and grades only for assigned offerings.
- Finance holds may restrict registration and related academic actions.
- Announcements, events, and notifications shall be attributable to authorized publishers.
- Audit logs shall preserve evidential traceability for critical actions.
- The system shall support a single-campus or logically unified institutional boundary in the initial phase.
- External integrations such as payment gateways and live email delivery may be deferred to later phases without changing the core domain architecture .